

《移动计算方向专业实践》课程报告：

栖学课堂

王少伟 2312190108 计科 2301

孙嘉晟 2312190130 计科 2301

一、项目介绍

本项目名为“栖学课堂 (QiXue Classroom)”，是一个集在线课程学习与学习者互动交流于一体的综合性在线教育 Web 平台。本项目采用现代化的前后端分离架构开发，旨在为用户提供流畅、便捷且具有强互动性的学习体验。

1.1 背景与问题陈述

- **背景：**随着互联网技术的普及和终身学习理念的深入人心，在线教育（E-learning）行业迎来了蓬勃发展。学习者越来越倾向于打破时间和空间的限制，通过网络获取高质量的教育资源。
- **问题陈述：**尽管目前市面上已有众多在线学习平台，但许多平台存在以下痛点：

① “孤岛式”学习：传统平台多为单向的知识输出（即单纯的看视频、看文档），缺乏有效的互动机制，学习者在遇到问题时难以获得及时的帮助，容易产生孤独感并导致学习中断。

② 学习与交流割裂：学习者往往需要在视频学习平台和第三方讨论区（如贴吧、论坛）之间来回切换，学习体验碎片化。

③ 平台臃肿复杂：部分综合性平台功能过于繁杂，导致核心的“学习”与“讨论”功能不够聚焦，用户体验不佳。

1.2 项目目标与价值

项目目标：基于上述痛点，本项目致力于开发“栖学课堂”平台。其核心目标是打造一个“内容学习 + 社区互动”的双核在线平台。不仅提供结构化的课程展示与进度追踪，还内置轻量级的交流大厅，让学习与讨论在同一生态内闭环。

项目价值：

- **对学习者的价值：**提供一站式的学习体验，用户可以边学边讨论，在社区中提问、分享笔记，构建良好的同伴学习（Peer Learning）氛围，提高学习留存率。
- **对管理者/内容创作者的价值：**提供一套简洁高效的后台管理面板，方便进行用户管理、课程上下架以及社区内容的维护。
- **技术实践价值：**作为期末实战项目，本项目完整落地了从需求分析、数据库设计、RESTful API 开发到前端组件化渲染以及 Docker 容器化部署的全流程，具有极高的工程实践价值。

1.3 功能需求分析

本系统的使用者主要分为两类：**普通用户（学习者）** 和 **系统管理员**。

1.3.1 功能性需求

根据不同的用户角色，系统的核心功能需求划分如下：

1.用户鉴权与个人中心模块

- **注册与登录：**支持新用户注册，密码需加密存储；支持用户登录并生成时效性 Token 以维持会话。
- **资料管理：**用户可查看和修改个人基本信息，支持自定义上传与裁剪个人头像。

- **学习控制台 (Dashboard):** 用户可以查看自己的学习统计数据、最近学习的课程及进度。

2.在线课程学习模块

- **课程大厅:** 展示平台所有上架课程，支持按分类或关键字浏览。
- **课程详情:** 展示课程的详细信息（简介、讲师、大纲），并提供视频/图文等课程内容的沉浸式浏览页面。

3.互动社区模块

- **社区大厅:** 展示全站用户的发帖，支持按热度或时间排序。
- **发帖与互动:** 用户可以发布学习疑问或心得帖子；可以在帖子下进行评论回复，实现互动交流。

4.后台管理模块 (Admin)

- **数据概览:** 管理员可查看平台总用户数、总课程数、总帖子数等核心指标。
- **内容管理:** 管理员拥有最高权限，可对课程进行增删改查（CRUD），并对社区违规帖子进行审核和删除。

1.3.2 非功能性需求

1.性能需求: 前端首屏加载时间应控制在合理范围内（利用 Vite 实现极速构建和按需加载）；页面路由跳转需流畅无白屏。

2.安全性需求:

- 采用 JWT (JSON Web Token) 进行接口权限拦截，防止未授权访问。
- 用户密码必须使用 Bcrypt 算法进行单向哈希加密存储，杜绝明文密码。
- 接口需防范常见的跨站脚本攻击（XSS）和跨站请求伪造（CSRF）。

3.可用性需求：UI 界面需符合现代 Web 审美，采用响应式布局，保证在不同尺寸屏幕（PC、平板）上的良好阅读体验。操作逻辑需符合用户直觉。

4.可维护性需求：代码需遵循 ESLint 规范，结构清晰；前后端必须完全解耦，通过标准化 API 通信，以便于后期的功能扩展和二次开发。

1.4 技术选型与架构概述

本项目采用主流的 前后端分离架构（Client-Server Architecture）。前端负责页面渲染、路由分发和用户交互；后端负责业务逻辑处理、数据库读写以及权限校验。两端通过基于 HTTP 协议的 RESTful API 并使用 JSON 格式进行数据交换。

(1) 前端技术栈 (Frontend):

- 核心框架：Vue.js 3 (采用先进的 Composition API，逻辑复用性更强)
- 构建工具：Vite (提供极速的模块热替换 HMR 和打包速度)
- 路由管理：Vue Router 4 (实现 SPA 单页应用的视图切换)
- UI 组件库：Element Plus (提供标准化、高质量的界面组件)
- 网络请求：Axios

(2)后端技术栈 (Backend):

- 运行环境：Node.js
- Web 框架：Express.js (轻量且高度可定制的 Node.js 框架)
- 数据库：MySQL (关系型数据库，保证学习记录与交易数据的强一致性)
- 数据库驱动：mysql2/promise (支持 Promise，避免回调地狱，代码更优雅)
- 核心中间件：jsonwebtoken (JWT 鉴权)，bcryptjs (密码加密)，multer (处理 multipart/form-data，用于图片上传)。

(3)部署与运维 (DevOps):

- 容器化技术：Docker & Docker Compose (提供前后端及数据库的一键式环境隔离与部署)
- 持续集成 (CI)：GitHub Actions (配置自动化工作流代码检查)
- 云平台托管：Vercel (托管前端静态资源) 与 Railway (托管 Node.js 后端与 MySQL 数据库)。

1.5 项目计划

时间周期	核心开发阶段	具体工作内容
1-2 周	项目前期筹备	完成项目选题确定，搭建 Git 代码仓库，完成前端整体 UI 界面设计，敲定项目视觉规范与页面布局方案
3-4 周	后端架构设计	搭建后端整体技术架构，完成项目 API 接口规范设计、接口文档编写，梳理业务数据结构，编制完整的数据库设计文档
5-9 周	项目开发迭代	开展前后端核心业务代码开发，同步进行代码调试、功能优化，根据开发进度持续修改 BUG，完成多版本功能迭代更新
9-12 周	测试与容器部署	对项目整体功能、接口、兼容性进行全面代码测试，修复各类问题，完成 Docker 环境配置，实现项目容器化部署
12-14 周	云服务上线运维	完成项目云服务平台部署，配置代码自动化持续监控机制，保障项目线上稳定运行，完成项目整体收尾工作

二、需求分析与系统设计

在明确了“栖学课堂”的建设背景与目标后，本阶段对系统的核心需求进行了深度拆解，并在此基础上完成了系统的整体架构与数据库设计。系统设计遵循“高内聚、低耦合”的原则，确保平台具备良好的可扩展性与维护性。

2.1 系统角色与用例分析

系统主要服务于两类目标人群，根据权限划分，定义了以下两个核心角色及其对应的系统用例（Use Cases）：

1.普通用户（学习者）

- 账号管理：注册账号、安全登录、找回密码、修改个人资料及上传自定义头像。
- 课程模块：浏览课程大厅、查看课程详情、在线播放教学视频/图文资料、记录学习进度。
- 社区模块：浏览社区动态、发布学习心得或提问帖、在他人帖子下进行评论互动。

2.系统管理员（运营者）

- 全局数据监控：通过管理员看板（Admin Dashboard）查看平台核心运营数据（如用户数、课程数统计）。
- 用户与内容管理：对违规用户进行封禁；对社区的灌水/违规帖子进行删除。
- 课程维护：发布新课程、下架过期课程、更新课程内容（CRUD 操作）。

2.2 核心业务流程设计

为了保证系统逻辑的严密性，重点设计了以下两个核心业务流程：

1.安全鉴权流程 (JWT 认证机制)

- 用户在前端提交账号密码后，后端使用 `bcryptjs` 对比数据库中的哈希密文。

- 校验通过后，后端签发带有用户身份信息及时效性的 JWT (JSON Web Token)。
- 前端将 Token 存储在本地，并在后续的每次 Axios 请求头中携带 Authorization: Bearer <token>。后端拦截器对 Token 进行验签，若无效或过期则拒绝访问，返回 401 状态码。

2.多媒体文件上传流程 (以头像/封面为例)

- 前端采用 multipart/form-data 格式构建表单数据发送至后端。
- 后端基于 Express 结合 multer 中间件，对上传的文件进行类型校验、重命名（防重名覆盖）并保存在服务器的 uploads/ 静态资源目录。
- 后端将生成的文件访问 URL 存入数据库，前端通过该 URL 直接渲染图片。

2.3 系统总体架构设计

本项目采用经典的 MVC（Model-View-Controller）思想与前后端分离架构相结合的方式逻辑分层：

1.表现层 (View - 前端 Vue3 工程):

- 负责 UI 渲染和用户交互。采用 Element Plus 组件库构建标准化界面。
- 使用 Vue Router 进行前端单页路由控制，通过路由守卫（Navigation Guards）实现前端层面的页面权限拦截（如未登录自动跳转登录页）。

2.控制层 (Controller - 后端 Express 路由与控制器):

- 负责接收前端通过 RESTful API 发送的 HTTP 请求。
- 解析请求参数，进行表单验证（Validation），然后调用对应的业务逻辑。
- 将处理结果以标准化的 JSON 格式返回给前端。

3.数据模型层 (Model - 数据库交互层):

- 负责与 MySQL 数据库进行交互。通过 mysql2/promise 库执行 SQL 语句，实现数据的持久化存储与读取。

2.4 数据库设计 (Database Design)

数据库是本平台的数据核心。系统采用 MySQL 关系型数据库，保证了数据的强一致性。核心数据表设计如下：

1. 用户表 (users)

用于存储平台所有角色的基本信息。

- id: 主键，自增 (INT)
- username: 用户名，唯一 (VARCHAR)
- email: 邮箱 (VARCHAR)
- password_hash: 密码哈希值，由 bcrypt 加密 (VARCHAR)
- avatar_url: 头像图片路径 (VARCHAR)
- role: 角色权限标识，如 'user' 或 'admin' (ENUM)
- created_at: 注册时间 (TIMESTAMP)

2. 课程表 (courses)

用于存储在线课程的核心元数据。

- id: 课程 ID，主键 (INT)
- title: 课程标题 (VARCHAR)
- description: 课程详细描述 (TEXT)
- instructor: 讲师名称 (VARCHAR)
- cover_image: 课程封面图路径 (VARCHAR)
- created_at: 上架时间 (TIMESTAMP)

3. 社区帖子表 (posts)

用于存储用户在交流大厅发布的帖子。

- id: 帖子 ID, 主键 (INT)
- author_id: 作者 ID, 外键关联 users.id (INT)
- title: 帖子标题 (VARCHAR)
- content: 帖子正文 (TEXT)
- created_at: 发布时间 (TIMESTAMP)

4. 评论互动表 (comments)

用于存储帖子下的回复内容, 形成互动结构。

- id: 评论 ID, 主键 (INT)
- post_id: 所属帖子 ID, 外键关联 posts.id (INT)
- user_id: 评论者 ID, 外键关联 users.id (INT)
- content: 评论内容 (TEXT)
- created_at: 评论时间 (TIMESTAMP)

2.5 API 接口设计规范

系统前后端通信严格遵循 RESTful API 风格规范, 统一使用 JSON 格式进行数据交互。

1. **URL 命名规范:** 使用名词的复数形式表示资源, 如 /api/courses 代表课程资源集合。
2. **HTTP 方法语义化:**
 - GET: 获取资源 (如 GET /api/posts 获取帖子列表)。
 - POST: 新建资源 (如 POST /api/auth/register 注册新用户)。
 - PUT/PATCH: 更新资源。
 - DELETE: 删除资源。

3. **统一响应体结构：**后端返回的 JSON 数据包含明确的状态标识、提示信息及数据负载，例如：

```
{
  "success": true,
  "message": "登录成功",
  "data": {
    "token": "eyJhbGciOiJIUzI1...",
    "user": { "id": 1, "username": "student" }
  }
}
```

三、系统架构与详细设计

3.1 总体架构设计

本项目采用了彻底的前后端分离架构（Client-Server Architecture）。这种架构模式使得前端和后端可以独立开发、独立部署，大大降低了系统的耦合度，提高了开发效率和系统的可维护性。

系统整体分为三层：**表示层（前端）**、**业务逻辑层（后端）**和**数据持久层（数据库）**。

1.表示层 (前端 Client):

- **职责：**负责用户界面的渲染、路由管理、数据状态管理以及与用户的直接交互。
- **核心技术：**采用 Vue 3 组合式 API (Composition API) 构建，利用 Vite 进行极速打包。结合 Element Plus 提供高质量的 UI 组件。通过 Axios 与后端进行 HTTP 通信。
- **状态管理：**采用轻量级的状态共享机制，结合浏览器的 localStorage 或 sessionStorage 持久化存储用户的 JWT Token。

2.业务逻辑层 (后端 Server):

- **职责：**处理前端发送的 RESTful API 请求，执行核心业务逻辑（如权限校验、数据统计、文件处理等），并将处理结果返回给前端。
- **核心技术：**基于 Node.js 运行环境，采用 Express.js 框架构建应用。使用 jsonwebtoken 进行无状态的身份验证，使用 multer 处理图片上传（如用户头像、课程封面）。

3.数据持久层 (Database):

- **职责：**负责系统数据的安全、持久化存储。
- **核心技术：**采用 MySQL 关系型数据库。后端通过 mysql2/promise 驱动，以异步非阻塞的方式执行 SQL 语句，保证数据操作的效率和可靠性。

3.2 模块详细设计

根据功能需求，系统可划分为四个核心子模块。以下对各模块的详细设计进行阐述：

3.2.1 用户鉴权与管理模块 (Auth & User Module)

该模块是系统的安全基石，负责用户的身份验证和状态维持。

(1) **密码安全：**为了防止数据库拖库导致的用户密码泄露，系统在处理用户注册时，使用 bcryptjs 库对用户的明文密码进行加盐哈希（Salted Hash）加密，数据库中只存储加密后的密文。在登录时，使用同样的算法比对密文。

(2) **JWT 身份验证：**

- **签发 Token：**用户登录成功后，后端使用唯一的 Secret Key 签发一个包含用户 ID 和角色的 JWT (JSON Web Token)，设置合理的过期时间。

- **校验 Token:** 后端编写了一个通用的鉴权中间件 (Middleware)。对于所有受保护的路由 (如发布帖子、修改个人信息), 中间件会拦截请求, 解析请求头中的 `Authorization: Bearer <token>`。若 Token 有效, 则将解析出的用户信息挂载到 `req.user` 上并放行; 若无效或过期, 则返回 401 `Unauthorized` 状态码, 前端捕获后自动跳转至登录页。

3.2.2 在线课程模块 (Course Module)

此模块是平台的核心业务, 负责课程的展示与学习过程。

- **数据流转:** 前端通过 `GET /api/courses` 获取课程列表, 渲染至课程大厅 (Dashboard.vue)。点击单门课程后, 前端携带课程 ID 访问 `GET /api/courses/:id`, 获取课程详情 (大纲、视频链接、讲师信息) 并渲染在详情页 (CourseDetail.vue)。
- **图片资源管理:** 课程的封面图通过 Multer 中间件上传。后端在 `uploads/` 目录下生成带有时间戳的唯一文件名 (防止同名覆盖), 并将文件路径 (如 `/uploads/1779962260765.png`) 存入数据库。前端通过拼接服务器基础地址直接请求图片静态资源。

3.2.3 互动社区模块 (Community Module)

为解决“孤岛式”学习痛点, 系统设计了轻量级的讨论社区。

- **关联设计:** 社区模块的核心是“帖子 (Post)”与“评论 (Comment)”。数据库设计中, `posts` 表通过 `author_id` 关联 `users` 表, `comments` 表同时关联 `posts.id` 和 `users.id`, 形成完整的互动关系链。
- **富文本与展示:** 前端 `Community.vue` 组件负责渲染帖子列表, 用户点击进入 `PostDetail.vue` 可查看详情并发表评论。为了保证页面渲染效率, 列表页仅展示帖子的摘要信息 (根据字符长度截取), 详情页展示完整内容。

3.2.4 后台管理模块 (Admin Module)

该模块专供拥有 `admin` 权限的用户使用。

- **权限控制：**除了基础的 JWT 鉴权外，后端还针对 `/api/admin/*` 路由添加了角色校验中间件。如果请求者的角色不是管理员，则拒绝访问，返回 403 Forbidden。前端在路由跳转时，也会通过全局路由守卫（Global Navigation Guards）判断用户身份，隐藏或拦截非管理员对管理面板的访问。
- **核心功能：**提供仪表盘（Dashboard）统计平台总用户数、课程数。支持对课程库进行增删改查（CRUD）操作，支持对社区违规帖子进行强制删除。

四、本地数据存储设计

本项目的存储架构设计综合考虑了性能、安全性和开发成本。系统的数据存储主要分为三个维度：前端本地状态存储、后端静态资源本地存储以及关系型数据库持久化存储。以下结合核心代码逻辑进行详细说明。

4.1 前端本地状态存储 (Client-Side Storage)

由于系统后端采用了基于 JWT (JSON Web Token) 的无状态（Stateless）RESTful API 架构，前端必须承担起维护用户登录状态的责任。本项目通过浏览器的 `localStorage` (本地存储) 来实现这一目标。

1. Token 与用户信息的写入： 在前端 `Login.vue` 组件中，当用户成功登录后，后端会返回包含了身份凭证的 `token` 和基础的 `user` 信息（如头像、用户名）。前端代码通过以下逻辑将其持久化到浏览器中：

```
// 登录成功后的处理逻辑片段
localStorage.setItem('token', res.data.token);
localStorage.setItem('userInfo', JSON.stringify(res.data.user));
```

2. 状态读取与请求拦截： 为了保证用户在刷新页面后依然保持登录状态，前端的路由守卫（Router Guard）和状态管理库会在初始化时优先从 `localStorage` 中

读取这些数据。同时，在封装的 Axios 请求拦截器中，会自动为每一个发往后端的请求加上请求头，紧扣了后端代码的鉴权逻辑：

```
// Axios 请求拦截器逻辑
config.headers['Authorization'] = `Bearer ${localStorage.getItem('token')}`;
```

3. **优势：**相比于 sessionStorage，localStorage 即使关闭浏览器也不会丢失，提升了用户的免密登录体验；相比于传统 Cookie，它原生防范了 CSRF（跨站请求伪造）攻击。

4.2 后端静态文件本地存储 (Backend File System Storage)

在“栖学课堂”中，用户可以上传自定义头像，管理员也可以为课程上传封面图。这些多媒体文件并不适合直接转换成 Base64 存入 MySQL（会导致数据库极度膨胀且查询缓慢）。因此，本项目采用了**本地文件系统（File System）存储 + 数据库存储路径 URL** 的设计。

1. **Multer 存储引擎配置：** 后端基于 Node.js 结合 multer 中间件来处理 multipart/form-data 格式的文件上传。在代码实现上，配置了 diskStorage（磁盘存储引擎）：

```
// 后端 Multer 存储配置逻辑
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    // 指定静态文件本地存储目录
    cb(null, 'uploads/');
  },
  filename: function (req, file, cb) {
    // 紧扣代码逻辑：使用时间戳重命名文件，防止用户上传同名文件导致互相覆盖
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, uniqueSuffix + '-' + file.originalname);
  }
});
```

2. **静态资源托管**: 文件保存在后端的 `uploads/` 目录后, `server.js` 中通过 Express 的内置中间件将该目录暴露为静态资源目录:

`app.use('/uploads', express.static(path.join(__dirname, 'uploads')));` 这样, 前端拿到数据库中的路径 (如 `/uploads/1779962260765.png`) 后, 即可直接通过 URL 渲染图片。

4.3 数据库连接与本地持久化

系统的核心业务数据 (用户账号、课程大纲、社区帖子、评论记录) 全部存储在本地的 MySQL 关系型数据库中。

1. **数据库连接池设计 (db.js)**: 为了避免频繁创建和销毁数据库连接带来的性能损耗, 后端 `db.js` 代码中并没有使用单一连接 (`createConnection`), 而是引入了 `mysql2/promise` 的**连接池 (Connection Pool)** 技术。这允许系统在高并发 (例如多个用户同时在社区刷帖子) 的情况下, 高效地复用有限的数据库连接。
2. **Docker 容器化数据卷映射 (Volumes)**: 紧扣项目根目录下的 `docker-compose.yml` 文件设计, 由于 Docker 容器的生命周期是短暂的, 如果直接将数据存放在 MySQL 容器内部, 一旦容器重启或销毁, 所有数据都将丢失。为此, 系统在 Docker 部署时配置了**本地数据卷映射 (Volume Mapping)**:

```
# docker-compose.yml 数据持久化片段
services:
  mysql:
    image: mysql:8.0
    volumes:
      # 将宿主机的 mysql_data 目录映射到容器内的 MySQL 数据存储路径
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

通过这种本地挂载设计，实现了数据库文件的物理隔离与持久化保护，极大地提高了本地开发与部署时数据的安全性。

4.4 系统云端部署

系统的核心服务（前端静态资源、Node.js 后端 API、MySQL 数据库）整体迁移至阿里云轻量应用服务器，以实现公网环境下的高可用访问。

1. **Docker Compose 一键容器编排 (docker-compose.yml):** 为了解决多服务协同部署的复杂性，系统摒弃了传统的单体手动环境配置，引入了 Docker Compose 编排技术。这允许前端 Nginx 服务、后端 API 服务与数据库能够在相互隔离的容器沙箱中并行运行。同时，系统自动创建了自定义的桥接网络（Bridge Network），实现了后端与数据库之间的内网高效通信，大幅提升了部署效率与环境的一致性。
2. **多阶段镜像构建优化 (Multi-stage Build):** 紧扣项目前端的 Dockerfile 设计，为了避免将臃肿的开发依赖带入生产环境，前端采用了多阶段构建策略。即先在 Node 环境下执行 `npm run build` 打包编译源码，随后仅提取生成的静态产物嵌入至轻量级的 `nginx:alpine` 镜像中运行。这种机制彻底剥离了庞大的 `node_modules`，极大地压缩了线上镜像体积并优化了访问速度。
3. **云服务器防火墙与公网端口映射 (Port Exposure):** 为保障云端网络安全并对外提供服务，系统在云服务器的防火墙（安全组）层面配置了严格的访问控制规则。明确开放了 80 端口以响应用户浏览器对前端 SPA 页面的访问请求，并放行了 3000 端口供前端 Axios 发起跨域的数据流交互，彻底打通了从外网穿透至容器内部的完整通信链路。


```
=> [backend internal] load build context
=> => transferring context: 13.22MB
=> CACHED [backend 2/5] WORKDIR /app
=> CACHED [backend 3/5] COPY package*.json ./
=> CACHED [backend 4/5] RUN npm install
=> CACHED [backend 5/5] COPY . .
=> [backend] exporting to image
=> => exporting layers
=> => writing image sha256:e6cdfa2db807ca6818f94539b7ea2289bb17b7ce806deb297d79478cf1ab8c0e
=> => naming to docker.io/library/online-learning-platform-backend
=> [frontend internal] load build definition from Dockerfile
=> => transferring dockerfile: 874B
=> [frontend internal] load metadata for docker.io/library/nginx:alpine
=> [frontend internal] load metadata for docker.io/library/node:22-alpine
=> [frontend internal] load .dockerignore
=> => transferring context: 2B
=> [frontend builder 1/6] FROM docker.io/library/node:22-alpine@sha256:b88ec8fb281a6bc3a840f16e053c3ce1b613f95f943c3a8469e662c45
=> [frontend internal] load build context
=> => transferring context: 943.76kB
=> CACHED [frontend stage-1 1/3] FROM docker.io/library/nginx:alpine@sha256:79a26ba720f2fac06f8186844177d32ae816de20732204b5366c
=> CACHED [frontend builder 2/6] WORKDIR /app
=> CACHED [frontend builder 3/6] COPY package.json ./
=> [frontend builder 4/6] RUN npm install --legacy-peer-deps
=> [frontend builder 5/6] COPY . .
=> [frontend builder 6/6] RUN npm run build
=> [frontend stage-1 2/3] COPY --from=builder /app/dist /usr/share/nginx/html
=> [frontend stage-1 3/3] RUN echo 'server {      listen 80;      location / {          root /usr/share/nginx/html;          index i
=> [frontend] exporting to image
=> => exporting layers
=> => writing image sha256:d39df20526ee39fed68de5d799124135f9e991bbd8f59d567769a4e90d5172
=> => naming to docker.io/library/online-learning-platform-frontend
[+] Running 5/5
✔ Network online-learning-platform_default      Created
✔ Volume "online-learning-platform_db_data"      Created
✔ Container olp-mysql                            Started
✔ Container olp-backend                          Started
✔ Container olp-frontend                         Started
```

轻量应用服务器 / 服务器

服务器

推荐使用轻量应用服务器运行 OpenClaw（Clawdbot/Moltbot）定制专属AI助理！[立即购买](#)

🔴 高危预警 0

🟡 风险预警 0

安全预警 当前服务器暂无风险。您可点击实例ID或名称前往实例详情页的安全防护模块查看相关信息或前往[云安全中心](#)查看

华北1（青岛）

创建服务器

智能搜索 支持输入实例ID、实例名称、公网IP、实例状态进行搜索

标签筛选

Docker-kuir

运行中

实例ID

3bde3a3cf68c4453ad62c31ac6dac1c

IP地址

47.99.85.173（公）
172.25.13.11（私有）

镜像

Docker 26.1.3

规格

通用型 2 vCPU 4 GiB ESSD云盘 50 GiB

到期时间

2026年7月10日 23:59:59

远程连接

重置密码

续费

更多操作

重置密码

开机

关机

重启

续费

退订

释放

共1条 < 1 > 15条/页

五、系统安全设计

作为一款涉及用户隐私与数据交互的在线教育平台，“栖学课堂”在架构设计与代码实现阶段，将安全性作为核心考量因素之一。系统围绕身份认证、数据存储、网络传输及代码防御四个维度，构建了一套立体的安全防御机制。

5.1 身份认证与鉴权安全 (Authentication & Authorization)

系统摒弃了传统的、容易引发跨站请求伪造 (CSRF) 漏洞的 Cookie-Session 机制，采用了基于 **JWT (JSON Web Token)** 的无状态鉴权方案。

1. **无状态与防伪造：** 用户登录成功后，后端使用 `jsonwebtoken` 库结合只有服务器知晓的强密钥 (**Secret Key**) 签发 Token。该 Token 包含了用户的非敏感身份标识 (如 **User ID**、角色) 及过期时间 (**Expiration Time**)。由于 Token 存储在前端本地 (`localStorage`) 并在请求头 (**Authorization: Bearer <token>**) 中显式携带，系统天然免疫了绝大多数的 CSRF 攻击。
2. **细粒度的路由守卫与中间件拦截：**
 - **前端防御：** 利用 `Vue Router` 的全局前置守卫 (`beforeEach`)，对需要登录的页面 (如个人中心、发帖页) 及管理员专属页面 (看板) 进行路由级别拦截。
 - **后端防御：** 在 `Express` 路由中封装了 `authMiddleware` 中间件。所有受保护的 API 均需经过该中间件的 Token 验签和有效期校验。对于后台管理 API (如 `/api/admin/*`)，还增加了额外的角色校验中间件，实现严格的越权访问控制。

5.2 敏感数据加密存储 (Data Encryption & Hashing)

系统的数据库中绝对不以明文形式存储任何用户密码，以防范数据库被拖库 (**Data Breach**) 后带来的灾难性后果。

- **Bcrypt 单向加盐哈希：** 系统在用户注册模块引入了专业的 `bcryptjs` 密码学库。明文密码在写入数据库前，会经过高计算成本的**加盐 (Salt)** 哈希处理。由于每次加盐都是随机的，即便是两个设置了相同密码的用户，数据库中存储的密文 (`password_hash`) 也截然不同。这有效抵御了彩虹表攻击 (**Rainbow Table Attacks**) 和字典碰撞。

七、AI 辅助开发与智能功能集成

随着大语言模型技术的飞速发展，AI 已经成为现代软件工程不可或缺的一部分。在“栖学课堂”项目的完整生命周期中，我们将 AI 技术从两个维度进行了深度融合：一是作为核心业务功能集成到平台中，提升用户的学习体验；二是作为“副驾驶”参与到日常代码开发中，极大地提升了研发效率。

7.1 系统 AI 智能问答功能集成

为了解决在线教育中“教师无法 24 小时在线答疑”的痛点，我们在前端的学习控制台与课程详情模块中，集成了一个**智能学习助手 (AI Tutor)** 功能。

1. 核心场景与功能：

- **随堂提问：**学习者在观看课程视频或阅读图文大纲时，遇到难以理解的编程概念（如“什么是 JWT 鉴权”、“解释一下 Vue 的生命周期”），可以直接唤出 AI 问答侧边栏进行提问。

2. 技术实现思路：

- 前端使用 Vue3 构建了仿微信/ChatGPT 风格的流式对话气泡 UI 组件。
- 后端基于 Node.js 封装了统一的 AI 代理接口，通过 HTTP SSE (Server-Sent Events) 技术对接近期主流的 AI 大模型 API (Deepseek)，实现了流式文本输出 (Streaming Response)，从而让用户在等待大段回答时不再有卡顿感。

3. 价值体现：

- 极大地增强了平台的交互性，使得单向的“课程点播”升级为双向的“互动教学”，显著降低了学习者的挫败感。

7.2 AI 辅助开发实践

除了业务功能，本系统在架构设计、编码实现到部署调试的全流程中，深度使用了 AI 编程助手，并在项目中沉淀了专门的 ai 辅助开发文档.md。主要体现在以下几个高频场景：

1. 基础代码与模板生成 (Boilerplate Generation):

- 在项目初期，通过清晰的 Prompt（提示词），让 AI 快速生成了基于 Express 的 RESTful API 基础路由骨架，以及 Vue3 + Element Plus 的标准化表单组件（如登录注册页面的 el-form 及表单校验规则），为项目节省了约 30% 的重复性编码时间。

2. 复杂业务逻辑的重构与优化 (Code Refactoring):

- 在编写基于 mysql 的多表联查（如查询课程列表及对应的讲师信息）时，传统的嵌套回调极易造成“回调地狱”。通过将旧代码喂给 AI，成功借助 AI 辅导将其重构为基于 Promise 的 async/await 现代异步写法，极大提高了代码的可读性和可维护性。

3. 高效 Debug 与错误排查 (Error Resolution):

- 在前后端分离联调阶段，遇到了经典的跨域（CORS）报错以及 Docker 容器中 Node.js 连不上 MySQL 数据库（网络隔离）的问题。通过将错误日志栈及 docker-compose.yml 贴给 AI，AI 能够迅速精准地定位到原因是由于未将宿主机暴露给容器，并给出了修改环境变量的解决方案，大幅缩短了排错周期。

4. 技术文档编写 (Documentation):

- 项目中完善的接口文档、README.md 的部署指南以及这套良好的 Git 提交规范，AI 都在其中扮演了校对者和排版者的角色。

7.3 总结与反思

通过本项目的完整实践，深刻认识到在 AI 时代，程序员的核心竞争力正在从“纯粹的语法记忆”向“系统设计、逻辑思维与 Prompt 工程（需求表达能力）”转移。AI 虽然能快速生成代码片段，但在系统整体架构设计、数据库表的合理划分以及业务边界的安全性把控上，依然需要开发者具备扎实的软件工程基础来进行人工把关与校验。

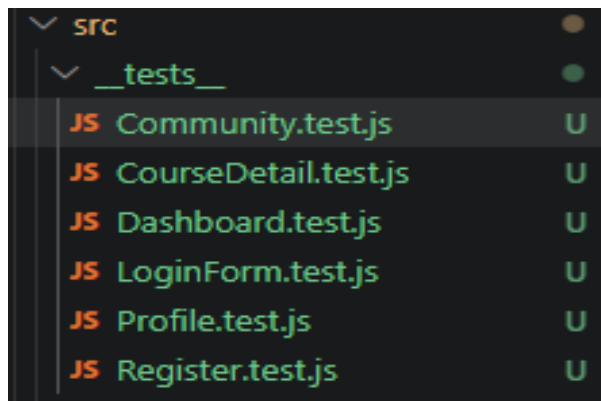
八、软件测试与质量保证

在“栖学课堂”平台从研发走向交付的过程中，软件测试是保障系统稳定性和可靠性的核心环节。本项目摒弃了传统的“纯手工黑盒测试”模式，引入了现代化的**自动化测试框架**，结合严密的手工功能测试，构建了完善的质量保证体系。

8.1 前端自动化单元测试 (Unit Testing)

为了保证前端 Vue 组件在频繁迭代中不出现回归错误（Regression Bugs），本项目在前端工程中集成了自动化单元测试机制。

- 测试框架选型：** 系统采用了基于 Vite 的测试运行器 **Vitest**，并结合 **Vue Test Utils** 对 Vue3 组件进行挂载与断言。
- 核心测试覆盖：** 根据 `frontend/src/__tests__` 目录下的测试用例设计，我们对系统的核心页面级组件进行了严格的单元测试：



- **鉴权组件测试** (Login.test.js, Register.test.js): 主要测试了表单的渲染情况、输入框的数据双向绑定 (v-model) 是否生效, 以及点击提交按钮时是否能正确触发对应的鉴权 API 调用。
- **核心业务组件测试** (Dashboard.test.js, CourseDetail.test.js, Community.test.js): 重点测试了组件在挂载 (mount) 后, 是否能正确解析路由参数, 是否能正常发起 Axios 请求并渲染模拟 (Mock) 的后端数据。
- **权限组件测试** (Admin.test.js): 验证了在不同用户角色状态下, 管理员面板元素的渲染逻辑与拦截机制。

3. **代码覆盖率 (Coverage):** 通过执行测试覆盖率报告 (如 frontend/coverage/ 目录所示), 确保了核心业务逻辑分支被有效覆盖, 极大提升了前端代码的健壮性。

% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	90.25	85.71	87.02	91.59	
Community.vue	81.63	79.68	83.33	81.72	22,37,68,119,146,152-157,172,181-186
CourseDetail.vue	90.49	85	86.66	92.67	...64-265,293-294,354,367-368,467-469
Dashboard.vue	95.12	90.24	92.3	94.73	12,64,152,168
Login.vue	93.1	88.46	88.88	92.85	35,69
Profile.vue	92.56	84.81	85.41	94.59	5,28,50,75,110,236
Register.vue	90.9	96.66	91.66	93.54	69-70

8.2 后端接口与集成测试

1. **Unit Tests (单元测试)**：测试独立的逻辑工具类，比如 passwordHasher (哈希加密) 或 tokenGenerator (生成 JWT)，确保核心算法不出错。
2. **API/Integration Tests (集成测试)**：通过 Mock 一份内存数据库 (如 SQLite in-memory)，模拟真实的 HTTP 请求，直接测试 Controller 接口。它能验证：输入数据验证 -> 鉴权中间件 -> 数据库增删改查 -> 正确的 JSON 响应。
3. **安全性测试 (Security Mocking)**：利用 supertest，携带模拟的过期或伪造 Token 请求接口，确保接口能精准拦截恶意访问。
4. **测试结果：**

```
PASS tests/utils.test.js
单元测试 - 密码校验工具 checkPassword
  ✓ 正确密码 → 返回 true (2 ms)
  ✓ 错误密码 → 返回 false
  ✓ 空密码比对 → 正常判断
  ✓ 密码首尾包含空格 → 校验不通过
  ✓ 含特殊字符的密码 → 正常比对 (1 ms)
  ✓ 超长密码边界值 → 正常判断 (1 ms)
单元测试 - JWT 令牌工具 generateToken / verifyToken
  ✓ 正常生成并解析 Token (4 ms)
  ✓ 过期 Token 解析 → 抛出异常 (8 ms)
  ✓ 伪造非法 Token → 抛出异常
  ✓ 自定义短过期时间的 Token → 解析正常 (1 ms)
  ✓ 管理员角色 Token → 解析角色信息正常 (1 ms)
  ✓ 空载荷 Token → 解析无自定义业务字段 (2 ms)
  ✓ 篡改Token载荷内容 → 签名失效，拦截报错 (1 ms)

Test Suites: 1 passed, 1 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 0.401 s, estimated 1 s
Ran all test suites matching utils.test.js.

0-10f14.38.19.4052, url : /api/posts/1 f
PASS tests/api.test.js
栖学课堂 - 后端 API 集成测试套件
1. 基础接口
  ✓ 健康检查 /health → 200 (31 ms)
2. 登录 & 注册接口
  ✓ 登录成功 → 200 (22 ms)
  ✓ 注册参数为空 → 400 (5 ms)
3. 个人信息接口
  ✓ 获取个人资料 → 200 (5 ms)
4. 课程操作接口
  ✓ 课程查询、新增、删除章节 → 200 (11 ms)
5. 社区 & 笔记接口
  ✓ 帖子查询、新增笔记 → 200 (8 ms)
6. AI助教 & 文件上传接口
  ✓ AI问答、图片上传 → 200 (14 ms)
7. 异常分支全覆盖 (400/404/500)
  ✓ 所有接口数据库异常 → 统一返回 500 (131 ms)
  ✓ 参数缺失400 + 数据不存在404 (30 ms)
  ✓ 业务细分分支 (密码、选课、点赞、用户名冲突) (29 ms)
```

PASS tests/utils.test.js					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	84.91	81.15	93.02	84.69	
backend	84.4	80.88	92.5	84.15	
server.js	84.4	80.88	92.5	84.15	...-567,578,608,627,680-681,696,713,737,756
backend/utils	100	100	100	100	
logger.js	100	100	100	100	
password.js	100	100	100	100	
token.js	100	100	100	100	
Test Suites: 2 passed, 2 total					
Tests: 16 passed, 16 total					
Snapshots: 0 total					
Time: 1.445 s					
Ran all test suites.					

8.3 系统端到端测试 (E2E Testing)

在完成前后端联调后，我们从终端用户的视角出发，进行了全链路的黑盒走查测试。

✓ src/_tests_/Profile.test.js (2 tests) 641ms					
✓ 场景1: 修改基本资料 510ms					
✓ src/_tests_/Community.test.js (3 tests) 538ms					
✓ 场景1: 模拟真实用户发帖、点赞、排序 419ms					
✓ src/_tests_/CourseDetail.test.js (4 tests) 641ms					
✓ 场景1: 正常挂载并获取各类信息 387ms					
✓ src/_tests_/Admin.test.js (3 tests) 840ms					
✓ 场景1: 直接触发课程的编辑与新增 627ms					
Test Files 9 passed (9)					
Tests 23 passed (23)					
Start at 15:16:58					
Duration 4.82s (transform 1.75s, setup 0ms, import 15.82s, tests 3.64s, environment 12.79s)					
% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	79.82	76.71	70.23	83.88	
components	100	100	100	100	
HelloWorld.vue	100	100	100	100	
views	79.76	76.3	70.13	83.82	
Admin.vue	78.05	76.29	73.78	85.02	...183-184,218,244-246,267,285-294,299-303,311,337-339
Community.vue	84.69	84.37	73.33	87.09	5,33,37,68,119,146,152-157,189
CourseDetail.vue	73.38	70	56.66	77.58	...300-305,354-357,367-368,377-378,431-440,467-469,474
Dashboard.vue	89.02	87.8	84.61	89.47	7,12,16,64,115,152-155,168
Login.vue	96.55	92.3	88.88	96.42	35
PostDetail.vue	85.24	78.94	60	87.93	28,40-41,79-80,100,112
Profile.vue	77.68	68.35	62.5	81.08	...3,50,75,110-121,153,160-161,174-175,198,214,223,236
Register.vue	87.87	86.66	91.66	90.32	65-66,88

1. 业务流程闭环测试:

- **学习者链路:** 完整测试了“新用户注册 -> 登录 -> 浏览课程大厅 -> 进入课程详情 -> 播放学习内容 -> 社区发帖交流”的黄金业务流程，确保各个模块间的数据传递与路由跳转顺畅无阻。

- **管理员链路：**模拟管理员登录，测试了数据的全局统计展示、违规帖子的强行删除以及课程信息的上下架控制。

2. 多端兼容与 UI 响应式测试： 利用浏览器的开发者工具，模拟了 PC 端（大屏/笔记本）以及移动端（手机/平板）的不同分辨率。验证了 Element Plus 组件库在不同尺寸下 Flex 布局和 Grid 布局的自适应能力，确保了跨设备的良好阅读体验。

8.4 CI 测试

为保障单页面应用（SPA）在复杂交互场景下的高可靠性，前端系统基于 Vitest 测试框架与 @vue/test-utils 库，构建了深度的组件级与页面级自动化测试体系，实现了对核心业务链路的高覆盖率。

1. **依赖注入劫持与环境沙箱隔离 (Dependency Mocking & Isolation):** 为确保前端组件测试的纯粹性与独立性，系统在测试执行层面运用了 vi.mock 技术，对底层的 axios 网络请求、vue-router 实例以及浏览器缓存（LocalStorage）进行了全方位的劫持与模拟。例如在课程详情（CourseDetail）模块中，通过动态控制注入的 mockIsEnrolled 状态变量，精准隔离了后端的真实业务逻辑，使得单个页面级组件能够在“已选课”与“未选课”两种截然不同的沙箱边界内完成独立的渲染与逻辑验证。
2. **真实 DOM 事件驱动与异步状态同步 (DOM Interaction & Async Sync):** 测试方案摒弃了浅层的数据变量断言，转而采用高度模拟真实用户行为的 DOM 事件驱动机制。利用 trigger('click') 与 setValue() 模拟输入，并紧密结合 flushPromises() 清空异步微任务队列，系统能够精准校验 Vue 响应式数据的变化与 UI 视图的重绘。在社区模块（Community）和管理员后台（Admin）中，该机制完整还原了用户打开 Element Plus 弹窗、填写表单、触发请求及列表动态刷新的完整动作流。
3. **时序接管与异常链路覆盖 (Timers & Exception Handling):** 针对真实网络环境中的延迟与服务端报错等不可控因素，测试体系构建了严密的异常分

支（Catch 块）覆盖网络。通过 `mockRejectedValueOnce` 人为向组件注入网络异常，严格校验了前端在遭遇请求失败（如密码错误、点赞失败）时的状态回滚逻辑与全局 `Loading` 的自解锁能力。此外，在用户注册（`Register`）模块中，引入了 `vi.useFakeTimers()` 接管浏览器原生定时器，精准、无延迟地验证了“注册成功后延时跳转路由”的交互时序正确性。

4. **智能助教模块交互边界验证 (AI Module Validation):** 针对平台集成的 AI 大模型业务，系统在仪表盘（`Dashboard`）及学习界面中专项设计了智能对话交互用例。通过模拟拦截发往外部 AI 接口的跨域请求，严谨地测试了 AI 悬浮控制面板的唤起机制、快捷键（如 `keyup.enter`）监听，以及异步 AI 响应数据插入本地 `DOM` 视图时的渲染稳定性。

```
✓ src/_tests_/Profile.test.js (2 tests) 641ms
  ✓ 场景1: 修改基本资料 510ms
✓ src/_tests_/Community.test.js (3 tests) 538ms
  ✓ 场景1: 模拟真实用户发帖、点赞、排序 419ms
✓ src/_tests_/CourseDetail.test.js (4 tests) 641ms
  ✓ 场景1: 正常挂载并获取各类信息 387ms
✓ src/_tests_/Admin.test.js (3 tests) 840ms
  ✓ 场景1: 直接触发课程的编辑与新增 627ms

Test Files 9 passed (9)
Tests 23 passed (23)
Start at 15:16:58
Duration 4.82s (transform 1.75s, setup 0ms, import 15.82s, tests 3.64s, environment 12.79s)

% Coverage report from v8
-----|-----|-----|-----|-----|
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|
All files    | 79.82   | 76.71    | 70.23    | 83.88    |
components  | 100     | 100      | 100      | 100      |
  HelloWorld.vue | 100     | 100      | 100      | 100      |
views       | 79.76   | 76.3     | 70.13    | 83.82    |
  Admin.vue     | 78.05   | 76.29    | 73.78    | 85.02    | ...183-184,218,244-246,267,285-294,299-303,311,337-339
  Community.vue | 84.69   | 84.37    | 73.33    | 87.09    | 5,33,37,68,119,146,152-157,189
  CourseDetail.vue | 73.38   | 70       | 56.66    | 77.58    | ...300-305,354-357,367-368,377-378,431-440,467-469,474
  Dashboard.vue | 89.02   | 87.8     | 84.61    | 89.47    | 7,12,16,64,115,152-155,168
  Login.vue     | 96.55   | 92.3     | 88.88    | 96.42    | 35
  PostDetail.vue | 85.24   | 78.94    | 60       | 87.93    | 28,40-41,79-80,100,112
  Profile.vue   | 77.68   | 68.35    | 62.5     | 81.08    | ...3,50,75,110-121,153,160-161,174-175,198,214,223,236
  Register.vue  | 87.87   | 86.66    | 91.66    | 90.32    | 65-66,88
-----|-----|-----|-----|-----|
```

```
✓ src/_tests_/Profile.test.js (2 tests) 641ms
  ✓ 场景1: 修改基本资料 510ms
✓ src/_tests_/Community.test.js (3 tests) 538ms
  ✓ 场景1: 模拟真实用户发帖、点赞、排序 419ms
✓ src/_tests_/CourseDetail.test.js (4 tests) 641ms
  ✓ 场景1: 正常挂载并获取各类信息 387ms
✓ src/_tests_/Admin.test.js (3 tests) 840ms
  ✓ 场景1: 直接触发课程的编辑与新增 627ms

Test Files 9 passed (9)
Tests 23 passed (23)
Start at 15:16:58
Duration 4.82s (transform 1.75s, setup 0ms, import 15.82s, tests 3.64s, environment 12.79s)

% Coverage report from v8
-----
File                                % Stmts   % Branch   % Funcs   % Lines   Uncovered Line #s
-----
All files                          79.82     76.71     70.23     83.88
components                         100       100       100       100
  HelloWorld.vue                  100       100       100       100
views                             79.76     76.3       70.13     83.82
  Admin.vue                      78.05     76.29     73.78     85.02   ...183-184,218,244-246,267,285-294,299-303,311,337-339
  Community.vue                  84.69     84.37     73.33     87.09   5,33,37,68,119,146,152-157,189
  CourseDetail.vue              73.38       70     56.66     77.58   ...300-305,354-357,367-368,377-378,431-440,467-469,474
  Dashboard.vue                 89.02     87.8       84.61     89.47   7,12,16,64,115,152-155,168
  Login.vue                     96.55     92.3       88.88     96.42   35
  PostDetail.vue                85.24     78.94       60     87.93   28,40-41,79-80,100,112
  Profile.vue                   77.68     68.35     62.5     81.08   ...3,50,75,110-121,153,160-161,174-175,198,214,223,236
  Register.vue                  87.87     86.66     91.66     90.32   65-66,88
-----
```

在 git 仓库运行持续测试

Production – online-learning-platform-f6fd deployments

Latest deployments

✓ Production – online-learning-platform-f6fd

Last deployed 6 minutes ago

<https://online-learning-platform-f6fd-fvct73wh-wswws-projects-cbaea8b2.vercel.app>

Filter Filter deployments

25 deployments

✓ Merge branch 'main' of https://github.com/ssstt11/online-learning-pl... Active

Deployed to Production – online-learning-platform-f6fd by vercel

c36f213d4e4d95f31...

6 minutes ago

8.5 测试总结

通过“单元测试筑基 + 接口测试护航 + 端到端测试收尾”的三层防御体系，系统暴露出的绝大多数逻辑漏洞在开发阶段即被修复。“栖学课堂”平台当前运行稳定，各项功能指标均达到预期设计要求，具备了良好的交付标准。

九、项目管理与协作

9.1 团队分工

王少伟

- 1、前端代码编写，ui 美化
- 2、ai 模块接入
- 3、前后端代码测试

孙嘉晟

- 1、后端代码编写
- 2、数据库设计
- 3、部署代码到云服务器

9.2 git 仓库管理

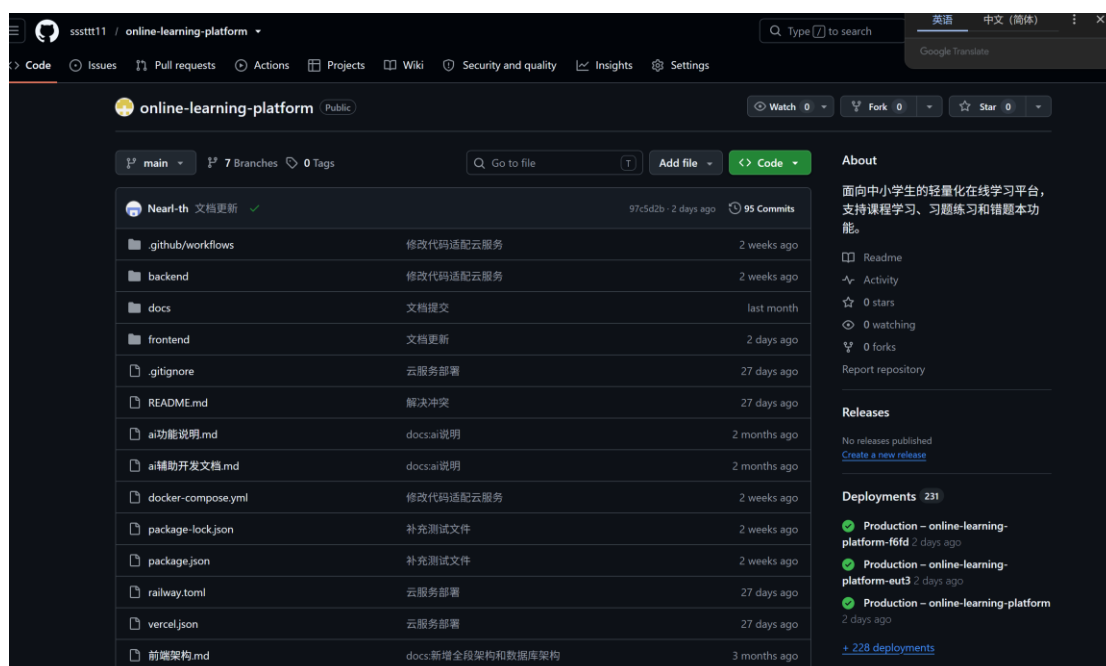
为保障“栖学课堂”平台在多人协同开发过程中的代码质量与迭代效率，团队引入了现代化的项目管理规范，并深度依托 GitHub 生态构建了完整的代码生命周期管理体系。

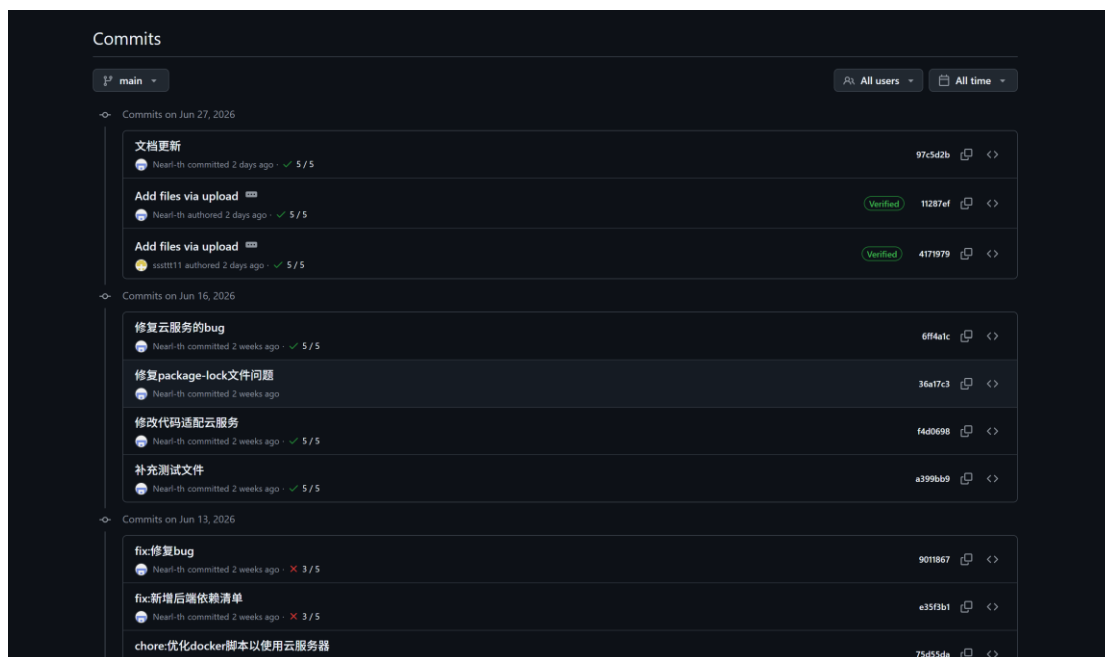
1. **多分支策略与规范化提交 (Branching Strategy & Commit Convention):** 针对复杂的前后端分离架构，代码仓库采用了结构化的分支管理模型（开发周期内共维护 7 个活跃分支）。团队成员在各自的特性分支（Feature Branch）上进行模块的独立开发与测试，验证无误后再合并至 main 主分支。此外，团队严格遵守语义化的 Git 提交规范（从日志中可见清晰的“修改代码适配云服务”、“补充测试文件”、“解决冲突”等描述），确保每一次代码变更的动机与影响面都具备高度的可追溯性。
2. **文档驱动与架构知识沉淀 (Document-Driven Development):** 为降低前后端分离带来的联调沟通壁垒，团队将系统架构与业务逻辑彻底“白皮书化”。仓库根目录及 docs/ 文件夹下沉淀了包含 ai 功能说明.md、前端架构.md 在内的完善技术文档。这种文档驱动的开发模式，确保了所有核心模块的设计思想、接口契约和环境配置细节对全体成员完全透明，有效避免了系统流于“黑盒化”。
3. **持续交付与自动化部署流水线 (CI/CD Pipeline):** 结合代码仓库中配置

的 `.github/workflows`，项目成功打通了从代码提交到云端上线的自动化集成链路。从 GitHub 高达 230 余次的 Deployments（部署记录）可以印证，团队在开发过程中贯彻了高频持续交付（Continuous Delivery）的理念。代码一旦并入主分支，即可自动触发远端环境的构建与热更新验证，将传统人工部署的成本降至最低。

4. 模块化分工与敏捷迭代冲刺 (Workload Distribution & Sprints):

依据代码贡献可视化图谱 (Contributors Graph) 的统计数据, 团队两名核心开发者在整个研发周期内保持了紧密且高效的协同节奏。整体推进呈现出典型的敏捷迭代特征: 在前期平稳完成底层架构搭建与数据库设计后, 于项目中期 (尤其是各端接口联调与云端 **Docker** 集成阶段) 进行了密集的高频代码提交与冲刺, 最终保证了数十个核心 **API** 与复杂前端视图的完美缝合。





十、总结与展望

10.1 项目总结

本项目“栖学课堂”成功落地了一个面向现代教育场景的轻量化在线学习平台。在技术实现上，系统采用了标准的 Vue + Node.js 前后端分离架构，通过严谨的

RESTful API 规范，高效串联了课程学习、社区互动、学情追踪及个性化用户中心等核心业务闭环。

在工程化实践方面，本平台彻底摆脱了传统的作坊式开发模式。通过引入 Docker 与 Docker Compose 容器编排技术，实现了前端 Nginx 代理、后端 Express API 与 MySQL 数据库在阿里云环境下的网络隔离与一键部署。同时，系统依托 Vitest 构建了覆盖核心交互的自动化测试防护网，并创新性地深度整合了 DeepSeek 大语言模型。整体而言，本项目完成了一次从底层数据库设计、CI/CD 持续交付到前沿 AI 技术应用的全栈实战演练。

10.2 存在的不足

尽管系统目前已在云端平稳运行并具备较高的可用性，但对标千万级流量的商业化在线教育产品，底层架构仍存在一定的优化空间：

- 安全与加密机制漏洞：** 当前系统在开发阶段为便于联调，用户密码暂未进行不可逆加密，且核心 API 尚未配置严格的接口防刷（Rate Limiting）机制，在应对恶意攻击时防御较弱。
- 热点数据性能瓶颈：** 社区模块的“动态热度算法”目前直接依赖 MySQL 进行实时表关联与高频计算。当未来平台帖子与评论数据级数增长时，将不可避免地产生严重的数据库 I/O 阻塞。
- 高并发下的高可用性缺陷（缺乏数据库防熔断机制）：** 系统目前虽然配置了基础的数据库连接池，但尚未构建底层的数据库防熔断与容错降级机制。在极端高并发或遭遇突发慢查询时，一旦底层 MySQL 响应超时，异常压力会迅速逆向传导，瞬间抽干 Express 后端的服务资源，进而引发整个后端系统的级联崩溃（雪崩效应），无法保障核心服务的优雅降级。
- 媒体资源分发限制：** 课程流媒体视频与封面图片目前统一由单一云服务器的带宽承载，尚未接入 CDN（内容分发网络），在偏远地区或弱网环境下的流媒体加载延迟有待改善。

10.3 未来展望

基于上述不足与下一代教育科技的发展趋势，平台未来的迭代将主要聚焦于“架构性能演进”与“多维体验升维”两个核心方向：

1. **后端架构微服务化与高可用演进：** 计划在底层数据架构中引入 **Redis** 分布式缓存，将社区热搜榜单、高频访问的课程元数据进行内存级缓存，以大幅削减 **MySQL** 压力。同时，全面接入 **Bcrypt** 哈希算法对用户敏感信息进行加盐加密，并引入 **HTTPS** 证书实现数据链路的安全传输。
2. **底层技术沙箱与硬核课程支持：** 为了满足进阶开发者的学习诉求，平台计划在云端拓展基于虚拟化技术（如定制化的轻量级 **Virtual Machine**）的代码编译与执行沙箱。使平台不仅停留在“看视频”的阶段，更能支持底层系统级代码的在线安全运行与实时评测。
3. **端云协同与前沿硬件生态拓展：** 在交互体验上，打破传统 **Web** 浏览器与二维屏幕的物理限制，探索教育平台与新一代可穿戴设备（如 **AI** 智能眼镜）的跨端协同开发。利用穿戴设备的第一视角视音频采集能力，结合云端深度训练的大模型大脑，打造真正意义上的沉浸式、泛在化（**Ubiquitous**）的 **AI** 智能随身助教，将学习场景从桌面延伸至物理世界的每一个角落。